



Debre markos University
Burie Campus
department of Computer Science
Web Programming
(CoSc3082)

Chapter Three Cascading Style Sheets
(CSS)



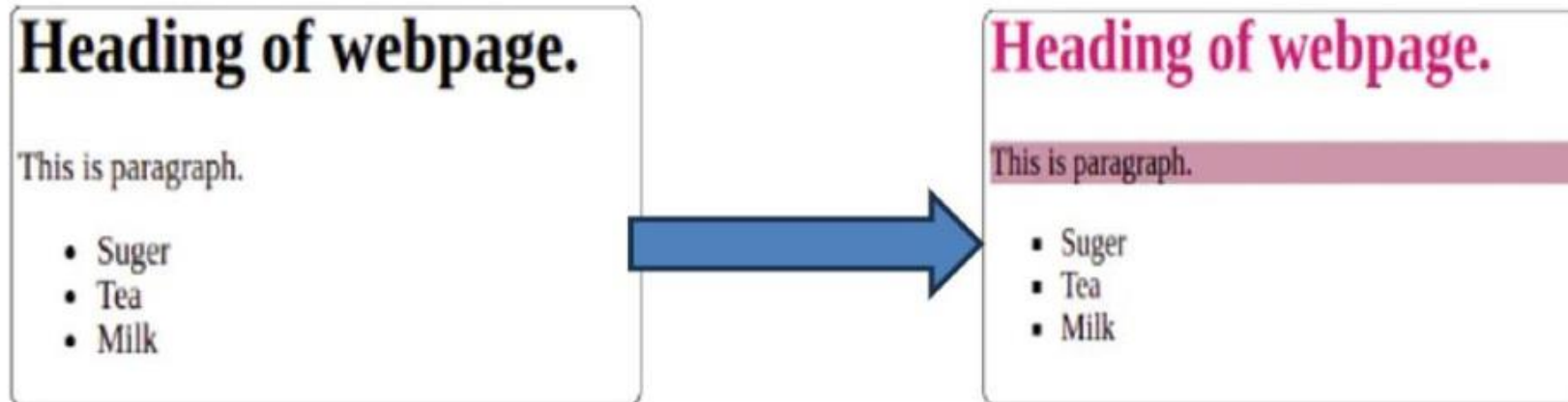
outline

Chapter 3: Cascading Style Sheet (CSS) (10hr)

- 3.1. CSS Basics
 - 3.1.1. Introduction to CSS
 - 3.1.2. CSS Syntax (CSS Selectors and Declarations)
 - 3.1.3. Attaching CSS with HTML (External , Embedded and Inline)
- 3.2. Style Sheet Rules
 - 3.2.1. Style Inheritance
 - 3.2.2. Style Rules Precedence
- 3.3. Style Properties
 - 3.3.1. Foreground and Background Properties
 - 3.3.2. Font and Text Properties
 - 3.3.3. CSS Box Model
 - 3.3.4. Table Styling Properties
 - 3.3.5. More On Styling List (Creating Navigation bars)
 - 3.3.6. Layout and Positioning Properties
- 3.4. CSS Measuring Units

Cascading Style Sheets

- CSS stands for “Cascading Style Sheets”
- CSS is used to style any webpage and make it look beautiful, structured, and responsive.
- It is used to add colors, fonts, sizes, and other styles to the website.
- CSS is a stylesheet language that describes the presentation of an HTML document.
- CSS describes how elements must be rendered on screen, on paper, or in other media.
- HTML is used to create the elements of a website like headings, paragraphs, images, Table, form, links, etc. But it doesn't provide any styling to the website.
- CSS to style the website.



What can be Done using CSS

Using CSS, you can control

- the color of the text, the style of fonts,
- the spacing between paragraphs,
- how columns are sized and laid out,
- what background images or colors are used, layout designs and many more....

Basic Syntax of CSS

- “**Style**” is a command that you set to tell the browser how a certain section of your webpage should look.
- `<style>` - an **HTML tag used to define a section that contains CSS.**
- You can use style on many HTML “elements” (like `<p>` `<h1>` `<table>` etc)
- Two parts: (1) *selector* and (2) *declaration*.

Selector: used to select the HTML elements you want to style.

`<p>` `<h1>` `<table>` etc

Declaration: This is **where the style rules are written**, enclosed in curly braces `{ }` it is Made up of *property* and *value*.

- **Property**: specifies the **attribute of HTML elements that we want to change.**
- ex: color, font, margins, boarder, etc.
- **Value**: the exact setting for that aspect.
- ex: red, italic, 40px, etc.



Example:

```
<style>
```

```
p {
```

```
  color: red;
```

```
  text-align: center;
```

```
}
```

```
</style>
```

```
<body>
```

```
<p>Hello World!</p>
```

```
<p>These paragraphs are styled with CSS.</p>
```

Hello World!

These paragraphs are styled with CSS.

CSS Selector

- **CSS selectors** are used *to select the content you want to style*.
- CSS selectors select HTML elements according to its id, class, type, attribute etc
- There are several different types of selectors in CSS.
 - a) CSS Element Selector
 - b) CSS Id Selector
 - c) CSS Class Selector
 - d) CSS Group Selector
 - e) CSS Universal Selector

The CSS element/type Selector

- ✓ The element selector selects elements based on the element name.
- ✓ Selects elements based on their tag name
- ✓ It is used to apply the same CSS declaration to all the elements with the same name.
- ✓ You can select all `<p>` elements on a page like this (in this case, all `<p>` elements will be center-aligned, with a red text color):

Example:

```
<style>
p {   color: green;
    text-align: center;
}
a {   color: red;
} </style></head>
<body> <h2>Element selector select all elements in the webpage.</h2>
<p>This is paragraph 1.</p>
<p>This is paragraph 2.</p>
<a href="#">Link 1</a>
<p>This is paragraph 3 and <a href="#">Link 2</a>.</p></body>
```

output

This is paragraph 1.

This is paragraph 2.

Link 1

This is paragraph 3 and Link 2.

The “id” selector

- The id selector is used to specify a style for a **single, unique** element.
- The id selector selects the HTML element with a unique identifier (id) and adds CSS to it.
- The id selector is specified using the hash (#) character, followed by the id of the element.
- The style rule below will be applied to the element with **id=“user”**:

```
<style>
```

```
#user{
```

```
  color: red;
```

```
  font-size: 20px;
```

```
} </style>
```

```
<body>
```

```
<p>Id selector selects unique element</p>
```

```
<p id="user">This paragraph has id "user".</p>
```

```
</body>
```

output

Id selector selects unique element

This paragraph has id "user".

The “class” selector

- The class selector selects HTML elements with a specific class attribute.
- It is used with a **period character . (full stop symbol) followed by the class name.**
- **Note:** A class name should not be started with a number.
- In the example below, all HTML elements with **class="center"** will be red and center-aligned:

HTML

```
<h1 class="myClass"> Hello Friends</h1>  
<h1> How are you    </h1>  
<h1 class="myClass"> How are you </h1>
```

CSS

```
.myClass{  
color: blue;  
}
```

Output

Hello Friends
How are you
How are you

CSS Class Selector con't

- You can also specify that only specific HTML elements should be affected by a class.
- With example below, only **<p>** elements with **class="center"** will be **center-aligned**:

```
<style>
```

```
p.center {
```

```
    text-align: center;
```

```
    color: red;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1 class="center">This heading will not be affected</h1>
```

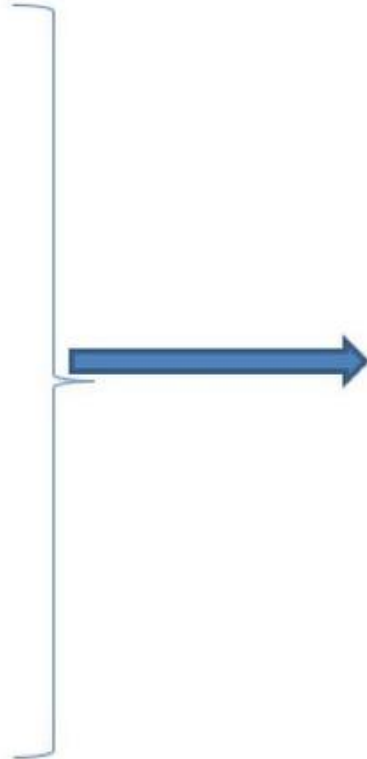
```
<p class="center">This paragraph will be red and center-aligned.</p>
```

```
</body>
```

CSS Group Selector

- The grouping selector is used to select elements that have the same style declarations..
- Grouping selector is used to minimize the code. Commas are used to separate each selector in grouping.
- Let's see the CSS code without group selector.

```
h1 {  
  text-align: center;  
  color: blue;  
}  
h2 {  
  text-align: center;  
  color: blue;  
}  
p {  
  text-align: center;  
  color: blue;  
}
```



It can be grouped in following ways:

```
<style>  
h1, h2, p {  
  text-align: center;  
  color: red;  
}  
</style>
```

The Universal Selectors

- Rather than selecting elements of a specific type, the universal selector quite simply matches the name of any element type.
- The **universal selector selects every single HTML element on the page.**
- It is written using the **asterisk (*) character.**

```
<style>
```

```
* {  
  color: green;  
  font-size: 20px;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>This is heading</h2>
```

```
<p>This style will be applied on every paragraph.</p>
```

```
<p id="para1">Me too!</p>
```

```
<p>And me!</p>
```

```
</body>
```


➤ CSS attribute selector

- The attribute selector is used to select the HTML element by its attribute name or attribute value.
- It is represented by the square brackets [] and the attribute name inside the square brackets.
- For example, [attribute].
- To select an element based on an attribute value write the attribute name in a square bracket and give its value in quotes [attribute="value"].
- Example:

```
<div > Text 1 </div>
```

```
<div myattr > Text 2 </div>
```

```
<div myattr class="aText" > Text 3 </div>
```

```
<div myattr class="aText" id="myId" > Text 4 </div>
```

```
<style>
```

```
body      { font-size: 10px; }
```

```
div       { font-size: 11px; }
```

```
[myattr]  { font-size: 12px; }
```

```
.aText    { font-size: 13px; }
```

```
#myId     { font-size: 14px; }
```

```
</style>
```

Attaching CSS with HTML (External , Embedded and Inline)

- Where do you put the styles?
- There are three ways to include CSS in HTML files.:
 - I. **Inline Style sheet:** Inline with individual tags
 - II. **Internal/Embedded Style sheet** :Internally to each page
 - III. **External Style Sheet:** Externally to the pages in a site, in a separate file

1) Inline Style

- Inline style is the CSS code that is **written within/directly** the HTML element itself.
- Inline style is the **approach of adding CSS rules directly** to the HTML element using the style attribute.
- Inline CSS has the **highest priority** out of external and internal CSS.
- This means that you can **override styles** that are defined in external or internal by using inline CSS.
- The inline element is **used by elements** by assigning **style attributes** to them and defining CSS property inside quotes as a **key-value pair**.
- Example: Example `<p style="color: blue;">Hello World!</p>`.

```
<body style="background-color:#182C61">  
<h4 style="color:#b5f3a8">Learning How to add inline CSS.</h4>  
<p style="color:white;font-size:20px">hellow world.</p></body>
```

Learning How to add inline CSS.

hellow world.

2) Internal Style Sheet

- This type of CSS is only for **Single Web Page**.
- Internal CSS is the **CSS code** that is written within the HTML file itself within the `<style>` element and is placed in the `<head>` section of the HTML file.
- The HTML code below contains an example of `<style>`'s usage.

```
<head> <style>
  body {
    background-color: #df8C61;  }
  h4 {  font-size: 35px;
        font-family: sans-serif;  }
  p {  font-size: 20px;
       color: white;  }
</style> </head>
<body>
  <h4>Adding CSS code.</h4>
  <p>Write internal style .</p>
</p>
</body>
```

Adding CSS code .

Write internal style .

3) External Style Sheet

- When using CSS it is preferable to keep the **CSS separate from your HTML**.
- Placing CSS in a separate file allows the web designer to completely differentiate between content (HTML) and design (CSS).
- External CSS is a file that contains **only CSS** code and is saved with a **".css"** file.

make a separate file and save it with the .css extension and write your CSS code in it.

Now to link this CSS file to your HTML file, you need to use <link> tag in the <head> section of your HTML file.

To connect a CSS file give the URL of the stylesheet by using href attribute and set rel="stylesheet".

Syntax:

```
<head>
```

```
<link rel="stylesheet" type="text/css" href="mystyle.css">
```

```
</head>
```


3) External Style Sheet (Cont.)

- Finally, the **href attribute** sets the path to the style sheet document itself.
- The **rel attribute** specifies the relationship between this page and the style sheet, and will most often be set to a value of stylesheet.
- The **type** informs the browser as to which type of file it can expect when it requests the style sheet from the server, and will always be **text/css**.

Demo.html

```
<html>
<head>
<link rel="stylesheet" type="text/css" href="test.css">
</head>
<body>
<p> Hello Friends </p>
<p id="para1"> How are you? </p>

</body>
</html>
```

test.css

```
#para1{
    text-align: center;
}
p
{
    color : blue;
}
```

Output

Hello Friends

How are you? 19

Including multiple external stylesheets

- You can also add multiple external style sheets to your HTML file using multiple `<link>` elements.
- CSS properties of all external stylesheets are added to the HTML file.
- `<head>`
- `<link rel="stylesheet" href="external-1.css">`
- `<link rel="stylesheet" href="external-2.css">`
- `<link rel="stylesheet" href="external-3.css">`
- `</head>`

CSS colors

- Color is one of the most important aspects of web design. It is used to make the website look more attractive and appealing.
- The **color property** defines the text color (foreground color in general) of an element.
- Colors in CSS most often specified in the following formats:
 - a **color keyword** - like "red", "green", "blue", "transparent", etc.
 - a **HEX value** - like "#ff0000", "#00ff00", etc.
 - an **RGB value** - like "rgb(255, 0, 0)"
- **Color Keywords**
- CSS defines the few color keywords which lets you specify color values in an easy way.

```
h1 {  
  color: red;  
}  
p {  
  color: purple;  
}
```

CSS colors

- Color is one of the most important aspects of web design. It is used to make the website look more attractive and appealing.
- The **color property** defines the text color (foreground color in general) of an element.
- Colors in CSS most often specified in the following formats:
 - a **color name** - like "red", "green", "blue", "transparent", etc.
 - a **HEX value** - like "#ff0000", "#00ff00", etc.
 - an **RGB value** - like "rgb(255, 0, 0)"
- **Color name**
- There are 147 predefined color names in CSS
- CSS defines the few color keywords which lets you specify color values in an easy way.

```
h1 {  
    color: red;  
}  
p {  
    color: purple;  
}
```

- Some of the most commonly used color names are:
- {red,green,blue,yellow,orange,purple,brown,black,white,gray}

CSS colors

- **HEX Color Values**

- Hex (short for Hexadecimal) is by far the most commonly used method of defining color on the web.
- Hex represent colors using a **six-digit code**, preceded by a **hash character**, like **#rrggbb**, in which **rr**, **gg**, and **bb** represents the **red**, **green** and **blue** component of the color respectively.

```
h1 { color: #ffa500; }  
p { color: #00ff00; }
```

RGB Color Values

- Colors can be defined in the **RGB model** (Red, Green, and Blue) using the **rgb()** functional notation.
- The **rgb() function** accepts **three comma-separated values**, which specify the amount of red, green, and blue component of the color.

```
h1 { color: rgb(255, 165, 0); }  
p {  
  color: rgb(0, 255, 0); }
```

- 3.2. Style Sheet Rules
 - 3.2.1. Style Inheritance
 - 3.2.2. Style Rules Precedence

Style Sheet Rules :Style Inheritance

- HTML elements can inherit CSS styles from their parent elements. This is called *CSS style inheritance*.
- Style inheritance in CSS means that *certain properties applied to a parent element can be passed down to its child elements*.
- This reduces the need to repeatedly apply the same styles to multiple elements.
- **Inheritable and Non-Inheritable Properties**
- *Not all CSS properties are inheritable.*

Style Sheet Rules :Style Inheritance

- **Inherited Properties:**

- Some CSS properties are inheritable, meaning their values can be passed down from parent elements to child elements.
- **Examples:** font-family, color, line-height, and text-align etc .

- **Non-Inherited Properties:**

- Other CSS properties are not inheritable and must be explicitly set for each element.
- These are known as non-inherited properties.
- Examples include **width, height, margin, and border**.
- style inheritance ensures that nested elements inherit certain properties from their parent elements unless explicitly overridden.

Example:Style Inheritance

<div>

<p> This text inherits the font-size of the parent div element. </p>

</div>

<style>

div {

font-family: Arial; }

</style>

- **div element** has the **font-family property** set to Arial.
- This CSS property is inherited by the **nested p element**, so it **will also** have its CSS property font-family set to Arial.

- HTML elements can **inherit styles from more remote ancestors too**, and not just from their parents.

<style>

body {

font-family: Arial; }

</style>

<body>

<div>

<p> This text inherits the font-size of the parent div.</p>

</div>

</body>

font-family is set on the **body** element. This CSS property is then **inherited by the div and p elements**.

Inheritable Properties

- color
- font-family
- font-size
- font-style
- font-weight
- line-height
- text-align
- text-indent
- text-transform
- visibility
- white-space
- word-spacing
- letter-spacing

Non-Inheritable Properties

- border
- margin
- padding
- width
- height
- background
- display
- position

- Note: styles that apply to **text** are **inherited**, whereas **borders, margins and paddings** and similar styles are **not**.

Controlling Inheritance:

- **The inherit Keyword**: Forces an element to inherit a specific property value from its parent using the **inherit** keyword.
 - This explicitly tells a property on a child element to inherit its value from the parent element.

```
.child { color: inherit; }
```

- **The initial Keyword**: Sets the property to its default value, which may or may not be inherited.

```
.child { color: initial; }
```

- **The unset Keyword**: Resets the property value to its inherited value if it is inheritable, or to its initial value if it is not inheritable.

```
.child { color: unset; }
```

Example

```
<head>
  <style>
    .parent {
      color: blue;
      font-size: 20px;
    }
    .child {
      font-size: 15px;
    }
  </style>
</head>
```

```
<body>
  <div class="parent">
    Parent text
    <div class="child">
      Child text
    </div>
  </div>
</body>
```

- The `.parent` class has a color of blue and a font-size of 20px.
- The `.child` class explicitly sets a font-size of 15px but does not set a color.
- The color property of `.child` will inherit the blue color from `.parent`, while the font-size will be 15px due to the specific declaration in `.child`.

Style rules precedence

- When multiple CSS rules target the same HTML elements, and those CSS rules set some of the same CSS properties, what styles end up being applied to the HTML element?
- **Style rules precedence** is a set of rules that determine how CSS are applied to an HTML document.
- When multiple CSS rules could apply to a single element, the precedence rules dictate which rule is followed.
- If there are two or more CSS rules that point to the same element, the selector with the highest specificity value will "win", and its style declaration will be applied to that HTML element.
- Here's a breakdown of the precedence rules, in order from highest to lowest:
 1. !important after CSS properties.
 2. Specificity of CSS rule selectors.
 3. Sequence of declaration.

Style rules precedence

1. !Important

If you need a certain CSS property to take precedence over all other CSS rules setting the same CSS property for the same HTML elements, you can add the instruction **!important** after the CSS property when you declare it.

The **!important** instruction has the highest precedence of all precedence factors.

- Here the `div {...}` CSS rule contains the instruction **!important** after the font-size CSS property, then that CSS property declaration takes precedence over all other declarations without the **!important** instruction targeting the same HTML element.

```
<style>
  div {
    font-family: Arial;
    font-size: 16px !important; }
  .specialText {
    font-size: 18px; }
</style>
<div class="specialText">
  This is special text.
</div>
```

Style rules precedence

2. specificity is a set of rules that determines which CSS property values are applied to an element.

- It is used to **resolve conflicts when more than one rule applies to a particular element.**
- It is a way to determine which style declaration has the highest priority.
- Specificity is based on the matching rules of CSS selectors.
- The more specific selector is the higher its priority.

CSS Selector	Description
CSS properties set directly on element, inside style attribute- Inline style.	Stronger specificity than ID selector.
ID	Higher specificity than class selector.
class	Higher specificity than attribute, element and universal selectors.
attribute	Higher specificity than element selector
element	Higher specificity than universal selector and inherited styles.
*	Lowest specificity of all directly targeted selectors
Inherited styles	Lowest specificity of all selectors - since an inherited style targets the element's parent, and not the HTML element itself.

3. Source Order: When specificity is equal, the last defined rule in the CSS file takes precedence.

Example :Style rules precedence

```
<body>
<style>
  body { font-size: 10px; }
  div { font-size: 11px; }
  .aText { font-size: 13px; }
  #myId { font-size: 14px; }
</style>
<div > Text 1 </div>
<div > Text 2 </div>
<div class="aText" > Text 3 </div>
<div class="aText" id="myId" >
Text 4 </div>
</body>
```

1. targets the **body element**; the **body element** in this CSS rule are inherited by the **div elements**.
2. targets the **div elements**. This CSS rule is more specific to div elements than the styles inherited from the body element.
3. targets **all HTML elements with the CSS class named aText**. The **CSS class selector** is more specific than a the **div element selector**.
4. targets **the HTML element with the ID myId**. The ID selector is more specific than the element selector, attribute selector and class selector, so the div element with the ID myId has this CSS rule applied.

3.3. Style Properties

3.3.1. Foreground and Background Properties

3.3.2. Font and Text Properties

3.3.3. CSS Box Model

3.3.4. Table Styling Properties

3.3.5. More On Styling List (Creating
Navigation bars)

3.3.6. Layout and Positioning Properties

foreground properties

- In CSS, **foreground and background properties** are essential for **controlling the appearance and layout of web elements**.
- They include a range of attributes that **define colors, images, positioning, and other styling aspects**.

Foreground properties

- Foreground properties primarily deal with the content within an element, such as text.
- Key foreground properties include:
 1. **color**: Sets the color of the text within an element. *p { color: blue; }*
 2. **font-family**: Specifies the font type for the text. *p { font-family: Arial, sans-serif; }*
 3. **font-size**: Defines the size of the text. *p { font-size: 16px; }*
 4. **font-weight**: Determines the weight (thickness) of the text. *p { font-weight: bold; }*
 5. **text-align**: Sets the horizontal alignment of the text. *p { text-align: center; }*
 6. **text-decoration**: Applies decorations to text, like underline, overline, or line-through. *p { text-decoration: underline; }*
 7. **line-height**: Controls the space between lines of text. *p { line-height: 1.5; }*
 8. **letter-spacing**: Adjusts the space between characters. *p { letter-spacing: 2px; }*

foreground vs background properties

Feature	Foreground	Background
Applies to	Text/icons	Area behind content
Main Properties	color, text-shadow, font	background-color, background-image
Overlap Handling	Text appears over the background	Background is behind content

CSS Font

- **CSS font** properties are **used to control the appearance of text**.
- The font properties **allow you to specify many aspects of the text**, such as the **font family**, **font size**, **font weight**, **font style**, and more.

- **Font Family**

- The font family of a text is set with the **font-family property**.
- Enclose multi-word font names in quotes

sans-Serif

- Arial, sans-serif:
- Helvetica, sans-serif:
- Gill sans, sans-serif:
- Lucida, sans-serif:
- Helvetica Narrow, sans-serif
- sans-serif

```
h1 { font-family:"sans-serif"; }  
h2 { font-family:"monospace"; }  
p { font-family:"Times New Roman"; }
```

Serif

Times, serif:
Times New Roman, serif:
Polatino, serif:
Bookman, serif:
New Century Schoolbook, serif
serif

CSS Font (Cont.)

- **Font Color**
 - Set the **text color** for different elements
- There are three different formats to define a color:
 - By **a color name**: color:red
 - By **hexadecimal value**:color: #9000A1;
 - By **RGB**: p { color:rgb(0, 220, 98); }
- **Font Size**
 - The **font-size property** sets the size of the text.
 - font-size : 120%
 - font-size : 10px;
 - font-size : x-large;

```
h4{  
    color : red;  
}
```

```
h4{  
    font-size: 120%;  
    font-size : 10px;  
    font-size : small;  
    font-size : smaller;  
    font-size : x-small;  
    font-size : xx-small;  
    font-size : large;  
    font-size : larger;  
    font-size : x-large;  
    font-size : xx-large;  
    font-size : medium;  
}
```

CSS Font (Cont.)

- **Font Style**

- The font-style property is mostly used to specify italic text.

```
h4{  
    font-style: italic ;  
}
```

- This property has three values:

- normal
- italic
- oblique

```
h4{  
    font-weight : 300;  
    font-weight : bolder;  
    font-weight : lighter;  
}
```

- **Font Weight**

- The font-weight property sets how thick or thin characters in text should be displayed.

```
h4{  
    font-variant: small-caps;  
}
```

- **Font Variant**

- The font-variant property specifies whether or not a text should be displayed in a **small-caps font**.

- font-variant : small-caps;

CSS Text Property

- While CSS Font covers most of the traditional ways to format your text, [CSS Text](#) allows you to control the [spacing, decoration, and alignment](#) of your text.
- CSS provides us the ability to style the text using its **text formatting properties**.
- These properties are used to change the appearance of the text.

property	description
text-align	alignment of text within its element
text-decoration	decorations such as underlining
line-height, word-spacing, letter-spacing	gaps between the various portions of the text
text-indent	indents the first letter of each paragraph

Text color

Text shadow

Text direction

Text indent

Text transform

White space

CSS Text Property (Cont.)

- **Text color**

- Color values can be specified in the following ways:
- Using color names like red, blue, green, etc.
- Using hexadecimal value like #ff0000, #0000ff, #00ff00, etc.
- Using RGB value like rgb(255, 0, 0), rgb(0, 0, 255), rgb(0, 255, 0), etc.
- Using HSL value like hsl(0, 100%, 50%), hsl(240, 100%, 50%), hsl(120, 100%, 50%), etc.

```
color: red;  
color: rgb(252, 186, 3);  
color: #03fc1c;  
color: hsl(220, 50%, 60%);
```

- **Text Indent**

- The `text-indentation` property is used to specify the indentation of the first line of a text.

```
h4{  
  text-indent : 20px;  
  text-indent : 30%;  
}
```

- **Text Align**

- The `text-align` property is used to set the horizontal alignment of a text.

```
h4{  
  text-align : right;  
  text-align : justify;  
  text-align : left;  
  text-align : center;  
}
```


CSS Text Property (Cont.)

- **Text Transform**
 - The `text-transform` property is used to specify uppercase and lowercase letters in a text.
- **White Space**
 - The white-space attribute allows you to prevent text from wrapping until you place a break `<br />` into your text.
- **Word Spacing**
 - With the CSS attribute word-spacing you are able to specify the exact value of the spacing between your words. `Word-spacing` should be defined with exact values.

```
h4{  
    text-transform : capitalize;  
    text-transform : uppercase;  
    text-transform : lowercase;  
}
```

```
h4{  
    white-space : nowrap;  
}
```

```
h4{  
    word-spacing : 10px;  
}
```


CSS Text Property (Cont.)

- **Letter Spacing**

- The **letter-spacing** property is used to set extra spacing between the characters of text/letter.

```
h4{  
    letter-spacing : 3px;  
}
```

- **Line Height**

- The **line-height** attribute will set the height of the line in the page.

```
h4{  
    line-height : 10px;  
}
```

CSS Background

- Background properties affect the space behind the content within an element.
- Using CSS **background** property we can set the background color, image, position, repeat, attachment, size of the background, etc.
- These properties include:

property	description
background-color	color to fill background
background-image	image to place in background
background-position	placement of bg image within element
background-repeat	whether/how bg image should be repeated
background-attachment	whether bg image scrolls with page
background	shorthand to set all background properties

Background Color

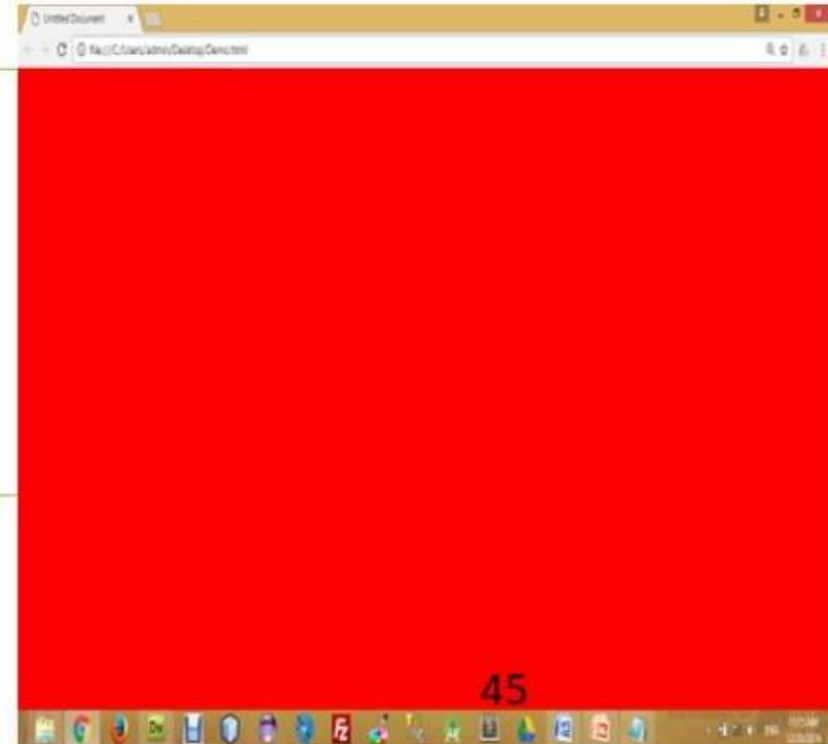
- The **background-color** property specifies the background color of an element.
- The background color of a page is defined in the body selector:
- The background color can be set to any color value, including **hex**, **RGB**, **RGBA**, **HSL**, **HSLA**, and **color keywords**.
- Below is example of CSS backgrounds

```
body  
{
```

```
    background-color : red;  
    background-color : #FF0000;  
    background-color : rgb(255,0,0);
```

```
}
```

test.css



Background Image

- The **background-image** property specifies an image to use as the background of an element.
- To set a background image, you must specify the URL of the image using the `url()` function. The URL must be enclosed in quotes.
- For Example,

test.css

```
body  
{  
  background-image :  
  url('pathToImage.jpg');  
}
```

Background repeat tutorials



Background Image Repeat

- When an image is set as a background image and the image is smaller than the element, the image is repeated to fill the entire element.
- You can have a background image repeat vertically (y-axis), horizontally (x-axis), in both directions, or in neither direction.

background-repeat : repeat-y;



background-repeat : repeat-x;



background-repeat : no-repeat;



Fixed Background Image

- The **background-attachment** property sets whether a background image is **fixed** or **scrolls** with the rest of the page.
- For Example,

```
test.css

body
{
    background-image : url('pathToImage.jpg');
    background-repeat : no-repeat;
    background-attachment : fixed;
}
```

background-position

- The **background-position** property sets the starting position of a background image.

background-position: <position>;

test.css

```
Body {
    background-image : url('pathToImage.jpg');
    background-repeat : no-repeat;
    background-position: 20px 10px;
    background-position: 30%30%;
    background-position: right top;
}
```

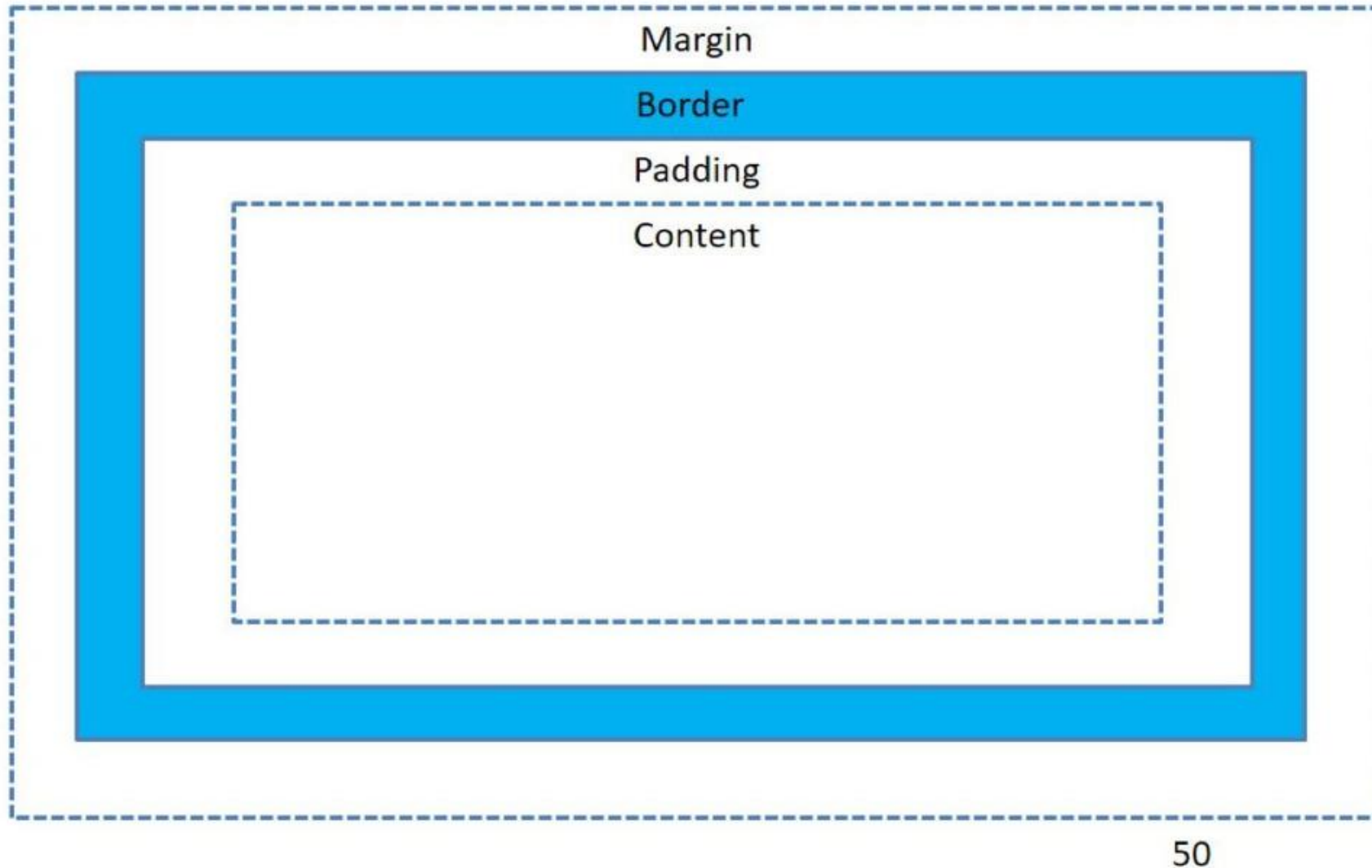
- **used to specify the initial position of a background image within an element.**
- This property allows you to control where the background image appears in relation to the element's content.

The Box Model

- **Every element** that can be displayed on a web page is comprised of one or more **rectangular boxes**.
- CSS box model typically describes **how these rectangular boxes** are laid out on a web page.
- These boxes can have different properties and can interact with each other in different ways, but every box has a **content area** and optional surrounding **padding, border, and margin areas**.
- In CSS, the term "box model" is used when talking about **design and layout**.
- The CSS box model is a fundamental concept that defines how the element's dimensions and spacing are calculated and it consists of: **margins, borders, padding, and the actual content**.
 - **CSS Box Model** in CSS that **defines how elements are displayed and how space is calculated around them**.

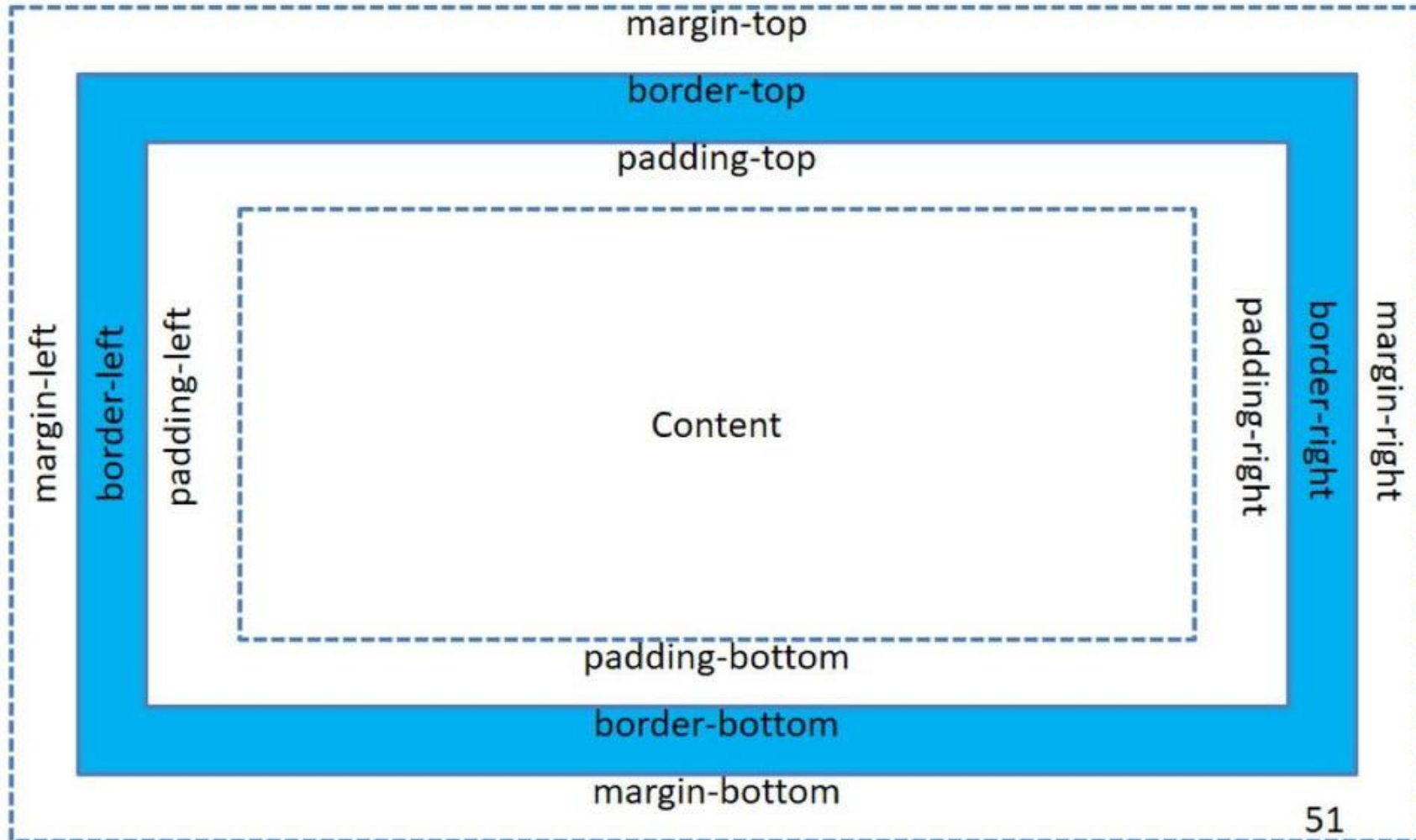
The Box Model (Cont)

- The image below illustrates the box model:



The Box Model (Cont)

- **The components of the box model are:**
- **content:** actual text or image that is displayed in the element



CSS Padding: Internal Spacing

- used to create space around the element's content.
- The CSS padding properties define the space between the element border and the element content.
- The top, right, bottom, and left padding can be changed independently using separate properties.
- A shorthand padding property can also be used, to change all padding at once.

```
h4{  
    padding : 10px;  
}
```

```
h4{  
    padding-top : 10px;  
    padding-right : 20px;  
    padding-bottom : 30 px;  
    padding-left : 40 px;  
}
```

```
h4{  
    padding : 10px 20px 30px 40px;  
}
```

padding (shorthand)

- The padding property is used to set the padding on all four sides of an element.
- This is a shorthand property for padding-top, padding-right, padding-bottom, and padding-left properties.
- It accepts 4 different sets of values.
- **If the padding property has four values**
- padding: 25px 50px 75px 100px;
 - top padding is 25px
 - right padding is 50px
 - bottom padding is 75px
 - left padding is 100px
- **If the padding property has three values:**
- padding: 25px 50px 75px;
 - top padding is 25px
 - right and left paddings are 50px
 - bottom padding is 75px

- **If the padding property has two values:**

- padding: 25px 50px;
 - top and bottom paddings are 25px
 - right and left paddings are 50px

- **If the padding property has one value:**

- padding: 25px;
 - all four paddings are 25px

CSS Border:

- The border is the line that surrounds the padding and content of the element.
- The CSS border properties allow you to **specify the style and color of an element's border**.
- **Border Style Types**
 - The **border-style property** specifies what kind of border to display.
- The following values are allowed:
- **dotted** - Defines a dotted border
- **dashed** - Defines a dashed border
- **solid** - Defines a solid border
- **double** - Defines a double border
- **groove** - Defines a 3D grooved border. The effect depends on the border-color value
- **ridge** - Defines a 3D ridged border. The effect depends on the border-color value
- **inset** - Defines a 3D inset border. The effect depends on the border-color value
- **outset** - Defines a 3D outset border. The effect depends on the border-color value

```
p.dotted {border-style: dotted;}  
p.dashed {border-style: dashed;}  
p.solid {border-style: solid;}  
p.double {border-style: double;}  
p.groove {border-style: groove;}  
p.ridge {border-style: ridge;}  
p.inset {border-style: inset;}  
p.outset {border-style: outset;}  
p.none {border-style: none;}  
p.hidden {border-style: hidden;}  
p.mix {border-style: dotted dashed  
solid double;}
```

A dotted border.

A dashed border.

A solid border.

A double border.

A groove border.

CSS Border:

- **Border Width**
 - The border-width property is used to set the width of the border.
- **Border Color**
 - The border-color property is used to set the color of the border.
 - Border colors can be any color defined by RGB, hexadecimal, or key terms. Below is an example of each of these types.
 - **color name** - like red, green, blue, etc.
 - **HEX value** - like #a53427.
 - **RGB value** - like rgb(255, 165, 0).
 - **HSL value** - like hsl(9, 100%, 64%).
- The top, right, bottom, and left border can be changed independently using separate properties.

```
h4{  
    border-width : 7px;  
}
```

```
h4{  
    border-color : red;  
}
```

```
h4{  
    border-top : 1px solid red;  
}
```

Border - Individual Sides

- In CSS, there is property for specifying each of the borders (top, right, bottom, and left):

```
p {  
    border-top-style: dotted;  
    border-right-style: solid;  
    border-bottom-style: dotted;  
    border-left-style: solid;  
}
```

Border - Shorthand Property

As you can see from the examples above, there are many properties to consider when dealing with borders. To shorten the code, it is also possible to specify all the individual border properties in one property. The border property is a shorthand property for the following individual border properties:

border-width	
border-style (required)	
border-color	

```
p {  
    border: 5px solid red;  
}
```


CSS Margin: External Space

- The margin properties set the size of the white space OUTSIDE the border.
- **Margin** is the space between the border of the element and the border of the adjacent element.
- The specified space is transparent and has no background color.

- The **top, right, bottom, and left margin** can be changed independently using separate properties.
- A shorthand margin property can also be used, to change all margins at once.
- i.e., **margin-top, margin-right, margin-bottom and margin-left.**

```
h4{  
    margin: 10px;  
}
```

```
h4{  
    margin -top : 10px;  
    margin -right : 20px;  
    margin -bottom : 30 px;  
    margin -left : 40 px;  
}
```

```
h4{  
    margin : 10px 20px 30px 40px;  
}
```


■ CSS margin as shorthand Property

■ The margin property can be used as shorthand for specifying the margin of one to four sides of an element. For example,

- `margin: 10px; /* applies to all sides */`
- `margin: 20px 40px; /* applies 20px to top & bottom, and 40px to left & right */`
- `margin: 20px 40px 60px; /* applies 20px to top, 40px to left & right, and 60px to bottom */`
- `margin: 10px 20px 30px 40px; /* applies to top, right, bottom, and left respectively */`

Styling Links

- By default, links are underlined and displayed in the browser's default link color. However, you can customise link styles to better suit your website's design.
- Therefore styling the links properly is an important aspect of building a user-friendly website

Anchor/Link States

– The **four** links states are:

1. **a:link** - a normal, unvisited link
2. **a:visited** - a link the user has visited
3. **a:hover** - a link when the user mouse over it
4. **a:active** - a link the moment it is clicked

```
a:link{  
    color:#FF0000;  
    /*unvisited link*/  
}
```

```
a:visited{  
    text-decoration : none;  
    /*visited link*/  
}
```

```
a:hover{  
    color:#00FF00;  
    /*mouse over link*/  
}
```

```
a:active{  
    color:#0000FF;  
    /*selected link*/  
}
```

Table Styling Properties

- You can make your table look more attractive using [CSS](#). You can change the **border**, **padding**, **background color**, **font size**, **font color**, etc. of the table.
- Here are some of the CSS properties that you can use to style your table.
 1. Border
 2. cell-spanning
 3. Table Caption
 4. Text align
 5. Background

Table without border

Name	Age	Dept
Abebe	18	CS
hana	20	CS
almaz	20	IT
bezawit	19	IT

Table with border

Name	Age	Dept
Abebe	18	CS
hana	20	CS
almaz	20	IT
bezawit	19	IT

Table with border colapse

Name	Age	Dept
Abebe	18	CS
hana	20	CS
almaz	20	IT
bezawit	19	IT

■ Border in Table

- Tables do not have borders by default. You can add borders to the table using the **border property**.
- Select **table, th, and td** and apply the **border property**.

```
<style>
  table, th, td {
    border: 1px solid black } </style>
```

Table Border Properties

- **border:** Defines the border for the table, rows, or cells.
- **border-width:** Specifies the width of the border.
- **border-style:** Specifies the style of the border (e.g., solid, dashed, dotted).
- **border-color:** Specifies the color of the border.

Table Size

- The size of the table can be changed by using the width and height properties.
- For example,

```
table {
  border-collapse: collapse;
  border: 1px solid black;
  width: 80%;
  height: 170px;
}
```

Collapsing Table Borders

- There are two distinct models for setting borders on table cells in CSS: *separate* and *collapse*.
- In the **separate border model**, which is the default, each table cell has its own distinct borders, whereas
- in the **collapsed border model**, adjacent table cells share a common border.

```
<style>
  table {
    border-collapse: collapse; }
  table, th, td {
    border: 1px solid black;
  }
</style>
```

Name	Age	Dept
Abebe	18	CS
hana	20	CS
almaz	20	IT
bezawit	19	IT

separate border(default)

Name	Age	Dept
Abebe	18	CS
hana	20	CS
almaz	20	IT
bezawit	19	IT

collapsed border

■ Table Cell(td,th) Properties

- **padding:** Adds space inside the table cells and header.
- **text-align:** Aligns the text inside the cells (left, right, center).
- **vertical-align:** Aligns the content vertically inside the cells (top, middle, bottom).

■ Example:

```
td, th {  
    padding: 15px;  
    text-align: left;  
    vertical-align: middle;  
}
```

■ Table Header Properties

- **background-color:** Sets the background color of the table headers.
- **color:** Sets the text color of the table headers.
- **font-weight:** Sets the font weight of the table headers.

■ Example:

```
th {  
    background-color: #f2f2f2;  
    color: black;  
    font-weight: bold;  
}
```


Horizontal Alignment

- The **text-align property** sets the horizontal alignment (like **left, right, or center**) of the content in **<th> or <td>**.
- By default, the content of **<th>** elements are center-aligned and the content of **<td>** elements are left-aligned.

```
th {  
    text-align: left;  
}
```

Background color in column

- target a column by using the **nth-child()** selector.
- The **nth-child()** selector selects every element that is the **nth child**, of a specific type, of its parent.
- To select the first column, use **nth-child(1)** with the **td** selector.
- Example: **td:nth-child(1)** selects the first column of every row.

Name	Age	Dept
Abebe	18	CS
hana	20	CS
almaz	20	IT
bezawit	19	IT

■ Row and Column Properties

- **:nth-child(even)** and **:nth-child(odd)**: Used to style even and odd rows differently.
- **.hover**: Adds styles when the mouse is hovered over a table row or cell.
- Use the **:hover selector** on `<tr>` to highlight table rows on mouse over:

```
tr:nth-child(even) {  
    background-color: #f9f9f9;  
}  
tr:hover {  
    background-color: #f1f1f1;  
}
```

Name	Age	Dept
Abebe	18	CS
hana	20	CS
almaz	20	IT
bezawit	19	IT

■ Caption and Summary Properties

- **caption-side**: Specifies the position of the table caption.
- **caption**: Styles the table caption.

```
caption {  
    caption-side: top; /* or  
    bottom */  
    text-align: center;  
    font-weight: bold;  
    font-size: 1.5em;}
```


■ Table Layout

- Table layout is the way in which the table cells are arranged.
- There are two types of table layout.
 - Fixed layout
 - Auto layout
- By default, the table layout is auto.
- You can change the table layout using the `table-layout` property.
- **When the table layout is fixed**, the width of the table is set to the width of the table container. **The width of the columns is set to the width of the widest cell in that column.**
- **When the table layout is auto**, the width of the table is set to the sum of the widths of the columns. The width of the columns is set to the width of the widest cell in that column.

```
table {  
    table-layout: auto; /* or fixed */  
    width: 100%;  
}
```

66

■ More On Styling List (Creating Navigation bars)

- Every website needs a [navigation bar](#) to help visitors navigate around the website.
- Having easy-to-use navigation is important for any web site.
- With CSS you can transform boring [HTML menus](#) into [good-looking navigation bars](#).
- [Navigation Bar = List of Links](#)
- [Navigation bars](#) are created using [HTML Lists](#) combined with [CSS](#) to make it look more like a Menu with multiple options.
- This is generally accomplished by putting a top
 - [horizontal navigation bar](#) or a
 - [side vertical navigation bar](#).

[Home](#) [News](#) [Contact](#) [About](#)

properties to style list:

1. CSS list style type

- The **list-style-type property** is used to change the shape of the bullet point. It can be used with both ordered and unordered lists.
- The values that can be used with this property are:
 - disc , circle , square ,decimal , decimal-leading-zero – etc

2.CSS list style position

- The **list-style-position** property specifies whether the content of the list will be inside or outside of the bullet.
- The values that can be used with this property are:
- **inside** - Used to display the content inside the bullet.
- **outside** - Used to display the content outside the bullet.

Vertical Navigation Bar

- A simple list is a vertical navigation bar, if we make every list item a hyperlink

```
<li><a href="#home">Home</a></li>
```

```
<li><a href="#news">News</a></li>
```

```
<li><a href="#contact">Contact</a></li>
```

```
<li><a href="#about">About</a></li>
```

```
</ul>
```

- [Home](#)
- [News](#)
- [Contact](#)
- [About](#)

- The above list can be styled by adding a few simple CSS properties:
- Now let's remove the bullets and the margins and padding from the list:

```
ul {  
  list-style-type: none;  
  margin: 0;  
  padding: 0; }  
</style>  
</head>
```

[Home](#)
[News](#)
[Contact](#)
[About](#)

Vertical Navigation Bar

- To build a vertical navigation bar, you can style the `<a>` elements inside the list, in addition to the code above:

```
<style>
ul {
  list-style-type: none;
  margin: 0;
  padding: 0; }
li a {
  display: block;
  width: 60px;
  background-color: #dddddd;
}
</style>
```

[Home](#)
[News](#)
[Contact](#)
[About](#)

- change the background color of the links when the user moves the mouse over them:
/* Change the link color on hover */
li a:hover {
 background-color: #555;
 color: white;
}

display: block; - Displaying the links as block elements makes the whole link area clickable (not just the text),

Active/Current Navigation Link

- Add an "active" class to the current link to let the user know which page he/she is on:
- Example

```
a.active {  
    background-color: #4CAF50;  
    color: white;  
}
```

Horizontal Navigation Bar

- When we create a Horizontal navigation bar, the main question is to **how to convert a basic List which is vertical to a horizontal list.**
- There are two ways to create a horizontal navigation bar.
 1. **inline**
 2. **floating** list items.

Using display:inline

Use `display:inline;` to **remove the line break before and after every list item**, because by default list items have property `display:block;` added to them.

```
<style>
```

```
li {  
    display: inline;  
}
```

```
</style>
```

[Home](#) [News](#) [Contact](#) [About](#)

Horizontal Navigation Bar

Using float:left

- Another way of creating a Horizontal Navigation bar is by adding `float:left`; CSS property to all the list items.

```
li {  
    float: left;  
}
```

A horizontal navigation bar with a light gray background. It contains four links: "Home", "News", "Contact", and "About", each underlined and in blue text.

[Home](#) [News](#) [Contact](#) [About](#)

B. Top, Right, Bottom, Left

- These properties define the offset for positioned elements:
- **top: 10px;** - Moves the element 10 pixels down from the top.
- **right: 10px;** - Moves the element 10 pixels left from the right.
- **bottom: 10px;** - Moves the element 10 pixels up from the bottom.
- **left: 10px;** - Moves the element 10 pixels right from the left.

CSS layout and positioning properties

- CSS layout and positioning properties are essential tools for controlling the arrangement and placement of elements on a web page.
- Here's an overview of some common layout and positioning properties in CSS:
- **Layout Properties:** defines how an HTML element is displayed on the webpage.

A. Display:

- **display: block:** The CSS display block values take up the full width of the screen. These values place/start the content of each element in a new line and allow them to act as a separate block(stretches out to the left and right as far as it can).
 - Common block-level elements include `<div>`, `<p>`, and `<h1>-<h6>` ul etc.
 - Example : `.block-element { display: block; }`
- **display: inline:** inline do not start on a new line and only take up as much width as necessary.
 - elements are place side by side or within a line (horizontally).
 - Common inline elements include `<span>`, `<a>`, `img` and `<strong>`.
 - Example: `.inline-element { display: inline; }`

CSS layout properties

- **Display: inline-block :** The inline-block offers the property of both the values inline and block.
 - This value is used whenever we want to place our content side by side (inline) and also want to set the height, width, and margins of the element.
 - **Syntax :** display: inline-block;
- **display: none;** The CSS display none value removes the HTML element and also does not display any blank space at the place of the removed element.
 - The removed elements exist in an HTML structure but not be display on a web page.
- **display: flex;** Turns the element into a flex container, allowing for flexible layout of its child elements.
 - **Flexbox:** is a one-dimensional layout model, ideal for aligning items in a row or column.(Navbars, cards in a row)
- **display: grid;** Turns the element into a grid container, allowing for grid-based layout of its child elements.
 - **Grid** is a two-dimensional layout model, suitable for complex layouts.

CSS layout properties

B. Float: used to place an element on the left or right side of its container, and allow text and inline elements to wrap around it.

Syntax: float: none | left | right | initial | inherit;

Value	Description
left	Floats the element to the left side of its container.
right	Floats the element to the right side of its container.
none	Default value. The element does not float.
inherit	Specifies that the value of the float property should be inherited from the parent element.
inline-start	Floats the element to the left side of its container in left-to-right text, or to the right side in right-to-left text.
inline-end	Floats the element to the right side of its container in left-to-right text, or to the left side in right-to-left text.

CSS layout properties

■ Layout Properties

C. Box Model Properties

- **margin** - Sets the outer space of an element.
- **padding** - Sets the inner space of an element.
- **border** - Sets the border around an element.
- **width** - Sets the width of an element.
- **height** - Sets the height of an element.

D. Clear

- **clear: left;** - Prevents floating elements on the left side.
- **clear: right;** - Prevents floating elements on the right side.
- **clear: both;** - Prevents floating elements on both sides.

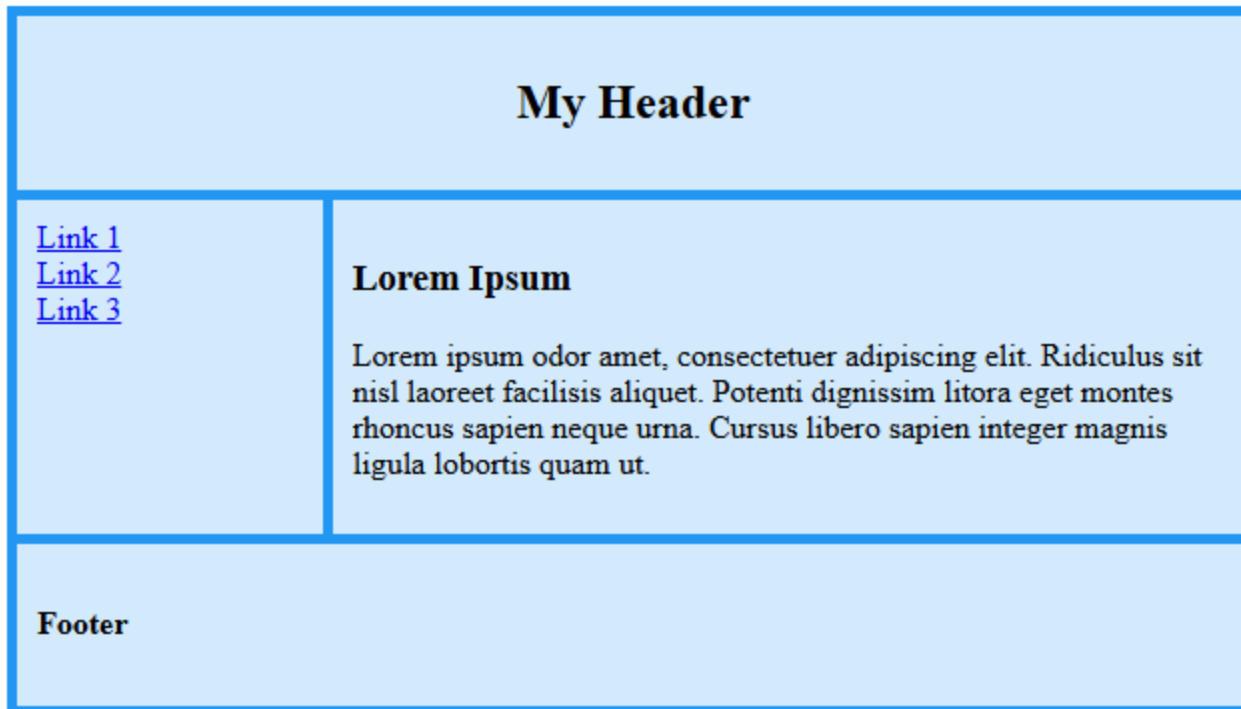
Layout in CSS

- refers to **how elements are arranged (structured and positioned) on a web page.**
- It determines the **visual structure**—how blocks, text, images, and sections are placed, sized, and aligned within the viewport or container elements.
- CSS provides different layout systems to control **where** and **how** elements appear.
- CSS provides several **layout techniques** to organize content effectively:

Layout Method	Used for
Normal Flow (Block and Inline Layout)	Default behavior where elements stack vertically (block) or flow horizontally (inline).
Flexbox	A one-dimensional layout system for aligning items in a row or column
Grid	A two-dimensional layout system for creating complex grid-based designs with rows and columns.
Positioning	Precise placement
Float	used for wrapping text around images, floats can create column-based layouts

CSS Grid Layout

- A grid always consists of:
- a **Grid Container** - defines the outer element of grid, where all the child elements are present. A grid container can be defined by setting `display: grid` or `display: inline-grid`
- **Grid Items** - are all the direct elements inside grid container. These items can be arranged vertically and horizontally inside container based on need



Display Grid Property

- A HTML element become a grid container if we set value of display property as `'grid'` or `'inline-grid'`.

- **grid**: This value defines block level grid container, means it behave like a block element taking full width available. (Similar to div)

```
.container {  
  display: grid;  
  grid-template-columns: auto auto auto;  
  background-color: dodgerblue;  
}
```

1	2	3
4	5	6
7	8	

become grid items.

- **inline-grid**: This value defines inline level grid container, means it behave like a inline element taking as much width as it's content. (Similar to span)

```
.container {  
  display: inline-grid;  
  grid-template-columns: auto auto auto;  
  background-color: dodgerblue;  
}
```

1	2	3
4	5	6
7	8	

Here's a table listing and describing key CSS Grid properties:

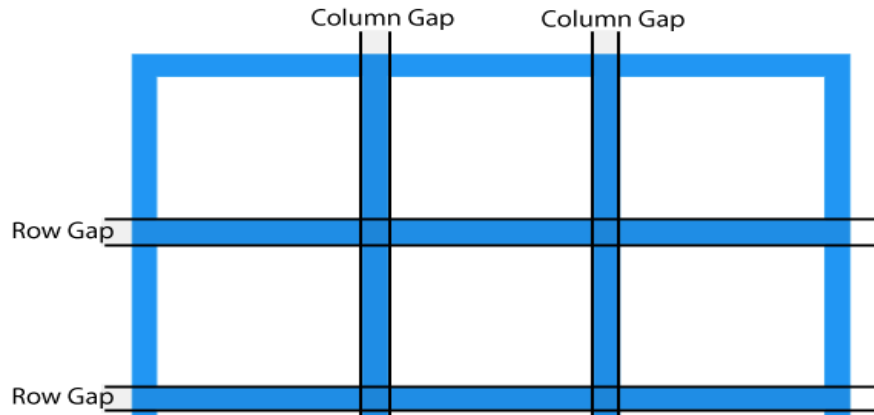
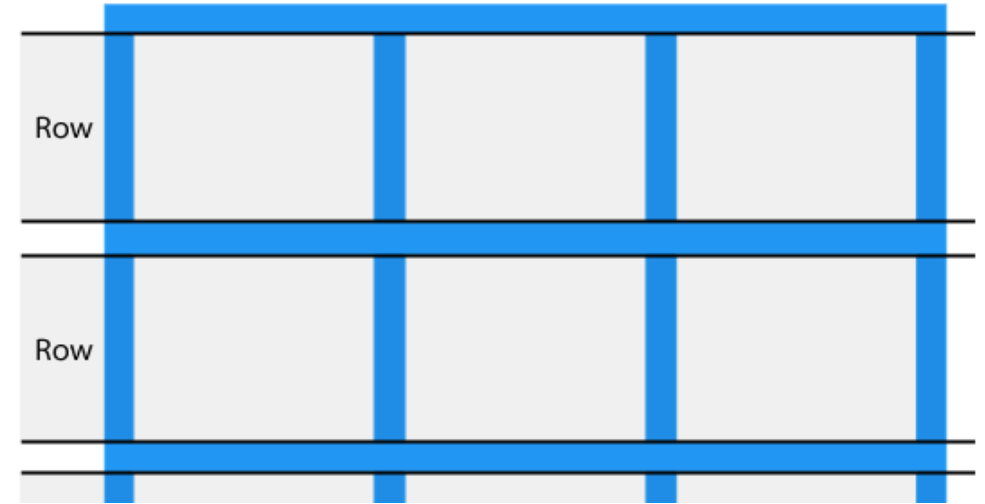
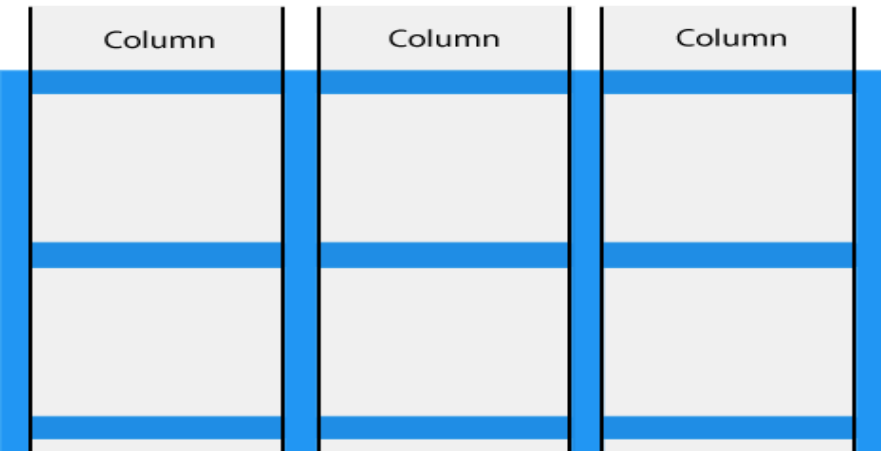
Property	Description
<code>display: grid;</code>	Enables grid layout on the container.
<code>grid-template-columns</code>	Defines the number and size of columns in the grid. You can use fixed sizes, percentages, or flexible units (e.g., fr).
<code>grid-template-rows</code>	Defines the number and size of rows in the grid, similar to columns.
<code>grid-template-areas</code>	Allows you to define named grid areas, making it easier to visualize and manage the layout.
<code>grid-column-start</code>	Specifies the line where a grid item starts in the column direction.
<code>grid-column-end</code>	Specifies the line where a grid item ends in the column direction.
<code>grid-row-start</code>	Specifies the line where a grid item starts in the row direction.
<code>grid-row-end</code>	Specifies the line where a grid item ends in the row direction.
<code>grid-column</code>	A shorthand for setting both <code>grid-column-start</code> and <code>grid-column-end</code> .
<code>grid-row</code>	A shorthand for setting both <code>grid-row-start</code> and <code>grid-row-end</code> .
<code>grid-gap</code>	Sets the size of the gaps (gutters) between rows and columns. Can also be defined separately with <code>grid-row-gap</code> and <code>grid-column-gap</code> .
<code>justify-items</code>	Aligns grid items along the row (horizontal) axis within their grid area.

Here's a table listing and describing key CSS Grid properties:

Property	Description
grid-column	A shorthand for setting both grid-column-start and grid-column-end.
grid-row	A shorthand for setting both grid-row-start and grid-row-end.
grid-gap	Sets the size of the gaps (gutters) between rows and columns. Can also be defined separately with grid-row-gap and grid-column-gap.
justify-items	Aligns grid items along the row (horizontal) axis within their grid area.
align-items	Aligns grid items along the column (vertical) axis within their grid area.
justify-content	Aligns the entire grid within the grid container along the row axis.
align-content	Aligns the entire grid within the grid container along the column axis.
grid-auto-columns	Defines the size of any implicitly created columns.
grid-auto-rows	Defines the size of any implicitly created rows.
grid-auto-flow	Controls how grid items are placed in the grid when they are not explicitly defined (e.g., row, column, dense).

CSS Grid Columns, Rows and Gaps

- The vertical lines of grid items are called *columns*.
- The horizontal lines of grid items are called *rows*.
- The spaces between each column/row are called *gap*



You can adjust the gap size by using one of the following properties:

- **column-gap**
- **row-gap**
- **gap**

Positioning properties

- The CSS **position property** is used to **set position** for an element.
- It determines how an element is positioned in the document and how it interacts with other elements.
- it is also **used to place an element behind another and also useful for scripted animation effect.**
- You can **position an element using the top, bottom, left and right properties.**
- These properties can be used only after position property is set first.
- Let's have a look at following CSS positioning:
 1. **CSS Static Positioning**
 2. **CSS Fixed Positioning**
 3. **CSS Relative Positioning**
 4. **CSS Absolute Positioning**

Positioning properties

- **position: static;:** The default positioning, which places the element in its normal position in the web page.
 - The static position is not affected by the top, right, bottom, and left values.

example:

positionStatic.html

<head>

<link rel="stylesheet" href=" positionStatic.css" />

<body>

<p>This is a normal paragraph.</p>

<p class="main">This paragraph is positioned using the static value.</p>

<p>This is a normal paragraph.</p>

</body>

positionStatic.css

```
p {  
  border: 1px solid black;  
  padding: 4px;  
}
```

```
p.main {  
  position: static;  
  top: 50px; /* doesn't work */  
  right: 50px; /* doesn't work */  
  bottom: 50px; /* doesn't work */  
  left: 50%; /* doesn't work */  
  background-color: greenyellow;  
}
```

Positioning properties

- **position: static;:** The default positioning, which places the element in its normal position in the document flow.
 - The static position is not affected by the top, right, bottom, and left values.
- example: **.element { position: static; }**
- **position: relative;:** Elements are positioned relative to their normal position. Setting the top, right, bottom, or left properties will move the element from its normal position.
- Example: **.element { position: relative; top: 10px; left: 20px; }**
- **position: absolute;:** The absolute value removes the element completely from its normal flow in the document.
- The element is positioned relative to their closest/nearest positioned parent element (an ancestor element with a position value other than static).
- Example: **.element { position: absolute; top: 50px; left: 100px; }**

Positioning properties

A. Positioning:

- **position: fixed:** **Elements** are positioned relative to the browser window.
 - Even if the page is scrolled, the element stays in the same place.
 - This is useful for creating sticky headers or footers.
 - Positions the element relative to the viewport, keeping it in the same place even if the page is scrolled.
 - **Example:** **`.element { position: fixed; top: 0; left: 0; }`**
- **position: sticky;** Combines relative and fixed positioning, allowing the element to switch between relative and fixed **positioning based on the user's scroll position.**
- **Example:** **`.element { position: sticky; top: 0; }`**

CSS measuring units

- CSS provides a variety of units for measuring length, width, height, and other dimensions.
- These units can be broadly categorized into absolute and relative units. Here's a detailed look at these CSS measuring units:

1. Absolute Units

- Absolute units are fixed and are not relative to any other element.
- They are generally used when a fixed size is needed.
 - **Pixels (px):** 1px = 1 pixel on the screen.
 - **Points (pt):** 1pt = 1/72 of an inch.
 - **Picas (pc):** 1pc = 12 points = 1/6 of an inch.
 - **Inches (in):** 1in = 2.54 cm = 96px.
 - **Centimeters (cm):** 1cm = 10 millimeters.
 - **Millimeters (mm):** 1mm = 1/10 of a centimeter.

CSS measuring units

2. Relative Units: is a unit of measurement that is *relative to another value*.

- The value is relative to the parent element.
- This means if the parent element is changed, the child element will change accordingly.

Em (em): Relative to the font-size of the element. For example, 2em means 2 times the current font size.

- **Rem (rem):**Relative to the font-size of the root element (usually the <html> element). For example, 1rem is equal to the root element's font-size.
- **Percentage (%):** Relative to the parent element's size. For example, width: 50% means 50% of the width of the parent element.
- **Viewport Width (vw):** 1vw = 1% of the viewport's width.
- **Viewport Height (vh) :**1vh = 1% of the viewport's height.

CSS measuring units

2. Relative Units:

- **Viewport Minimum (vmin):** $1\text{vmin} = 1\%$ of the viewport's smaller dimension (width or height).
- **Viewport Maximum (vmax):** $1\text{vmax} = 1\%$ of the viewport's larger dimension (width or height).
- **Character (ch):** Relative to the width of the "0" (zero) character in the element's font.
- **Ex (ex):** Relative to the x-height of the element's font. The x-height is the height of the lowercase "x" in the font.
- **Line Height (lh):** Relative to the line-height of the element.